

**Московский государственный технический
университет им. Н.Э. Баумана**

Факультет ИУ
Кафедра ИУ5

Курс «Парадигмы и конструкции языков программирования»

Отчет по домашнему заданию
Вариант №7

Выполнил:
студент группы ИУ5-32Б:
Гаранин Е. Д.
Подпись и дата:

Проверил:
преподаватель каф. ИУ5
Гапанюк Ю. Е.
Подпись и дата:

Москва, 2025 г.

ОГЛАВЛЕНИЕ

1. Задание для домашнего задания
2. Описание и листинг кода
3. Пример выполнения

ЗАДАНИЕ ДЛЯ ДОМАШНЕГО ЗАДАНИЯ

1. Выберите язык программирования и реализуйте на нем небольшой проект (с детальным текстовым описанием).
2. Проект может быть посвящен отдельному аспекту (аспектам) языка или содержать решение какой-либо задачи на этом языке.
3. Необходимо установить на свой компьютер компилятор (интерпретатор) этого языка и произвольную среду разработки.
4. В случае создания проекта необходимо детально комментировать код.
5. При создании проекта необходимо изучить и корректно использовать особенности парадигмы языка и основных конструкций данного языка.

ОПИСАНИЕ И ЛИСТИНГ КОДА

Выбор языка и платформы: В качестве языка программирования для реализации проекта был выбран **Kotlin**. Разработка велась под мобильную платформу Android. Выбор Kotlin обусловлен его лаконичностью, безопасностью и полной совместимостью с современными инструментами Android-разработки.

Архитектура приложения: Приложение спроектировано в соответствии с архитектурным паттерном **MVVM (Model-View-ViewModel)**, что обеспечивает разделение ответственности между компонентами, упрощает тестирование и поддержку кода.

- **Model:** Отвечает за получение и хранение данных. Включает в себя работу с локальной базой данных и сетью.
- **View:** Отвечает за отображение данных (Fragment, Activity).
- **ViewModel:** Служит связующим звеном, управляет состоянием UI и обрабатывает действия пользователя, используя StateFlow и Flow.

Реализация слоя данных

1. **Локальное хранение:** Для сохранения шуток на устройстве используется библиотека **Room**. Реализованы две сущности базы данных:
 - Joke – основная сущность для хранения добавленных шуток.
 - Cache – сущность для кэширования шуток, полученных из сети, с временной меткой.
 - Доступ к данным осуществляется через интерфейсы DAO (JokeDao, CacheDao), поддерживающие асинхронные запросы через Coroutines.
2. **Сетевое взаимодействие:** Загрузка случайных шуток реализована с использованием библиотеки **Retrofit**.
 - Интерфейс JokesApi описывает GET-запрос к публичному API, позволяя фильтровать контент и указывать тип шуток.
 - Ответы сервера парсятся в DTO-объекты (JokeResponse, JokesResponse) с помощью библиотеки Gson.
3. **Репозиторий:** Класс JokesRepository инкапсулирует логику работы с данными. Он реализует стратегию “Cache-First”: сначала проверяется наличие актуального кэша в БД, и только при его отсутствии или устаревании выполняется запрос к сети.

Реализация пользовательского интерфейса: Приложение построено по принципу **Single Activity**. Навигация между экранами осуществляется через замену фрагментов.

- **JokeListFragment:** Главный экран со списком шуток. Использует RecyclerView и JokeAdapter для эффективного отображения списка. Реализована пагинация (подгрузка при скролле).

- **JokeDetailsFragment:** Экран детального просмотра, куда передается ID шутки для отображения полного текста и метаданных.
- **JokeGenerateFragment:** Экран для ручного добавления новой шутки пользователем.

Бизнес-логика и управление состоянием

- **JokeViewModel:** Управляет состоянием экранов, используя MutableStateFlow для передачи статусов загрузки (LOADING, SUCCESS, ERROR) во View.
- **Асинхронность:** Все тяжелые операции (запросы к БД и сети) выполняются в фоновых потоках с использованием **Kotlin Coroutines**, что предотвращает блокировку интерфейса.

Cache.kt:

```
package com.example.jokeapplication.data.db.cache

import androidx.room.Entity
import androidx.room.PrimaryKey
import kotlinx.serialization.SerialName
import kotlinx.serialization.Serializable

@Serializable
@Entity(tableName = "cache")
data class Cache(
    @SerialName("id")
    @PrimaryKey
    val id: String,
    @SerialName("category")
    val category: String,
    @SerialName("question")
    val question: String,
    @SerialName("answer")
    val answer: String,
    val fromNetwork: Boolean,
    val timestamp: Long
)
```

CacheDao.kt:

```
package com.example.jokeapplication.data.db.cache

import androidx.room.Dao
import androidx.room.Insert
import androidx.room.Query
import kotlinx.coroutines.flow.Flow

@Dao
interface CacheDao {
    @Insert
    suspend fun insertCache(jokes: List<Cache>)

    @Query("SELECT * FROM cache WHERE timestamp > :timestamp")
    fun getCachedJokes(timestamp: Long): Flow<List<Cache>>
}
```

```

    @Query("DELETE FROM cache")
    suspend fun clearCache()
}

```

CacheDatabase.kt:

```

package com.example.jokeapplication.data.db.cache

import android.content.Context
import androidx.room.Database
import androidx.room.Room
import androidx.room.RoomDatabase

@Database(entities = [Cache::class], version = 1)
abstract class CacheDatabase: RoomDatabase() {
    abstract fun cacheDao(): CacheDao

    companion object {
        @Volatile
        lateinit var INSTANCE: CacheDatabase

        fun initDatabase(context: Context) {
            val instance = Room.databaseBuilder(
                context.applicationContext,
                CacheDatabase::class.java,
                "cache_database"
            ).build()
            INSTANCE = instance
        }
    }
}

```

JokeDao.kt:

```

package com.example.jokeapplication.data.db

import androidx.room.Dao
import androidx.room.Insert
import androidx.room.OnConflictStrategy
import androidx.room.Query
import com.example.jokeapplication.data.Joke
import kotlinx.coroutines.flow.Flow

@Dao
interface JokeDao {
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insert(joke: Joke)

    @Query("SELECT * FROM jokes")
    fun getAllJokes(): Flow<List<Joke>>
}

```

JokeDatabase.kt:

```

package com.example.jokeapplication.data.db

import android.content.Context
import androidx.room.Database
import androidx.room.Room
import androidx.room.RoomDatabase

```

```

import com.example.jokeapplication.data.Joke

@Database(entities = [Joke::class], version = 1)
abstract class JokeDatabase: RoomDatabase() {
    abstract fun jokeDao(): JokeDao

    companion object {
        @Volatile
        lateinit var INSTANCE: JokeDatabase

        fun initDatabase(context: Context) {
            val instance = Room.databaseBuilder(
                context.applicationContext,
                JokeDatabase::class.java,
                "joke_database"
            ).build()
            INSTANCE = instance
        }
    }
}

```

JokeApiState.kt:

```

package com.example.jokeapplication.data.network

data class JokeApiState(val status: Status) {

    companion object {

        fun success(): JokeApiState {
            return JokeApiState(Status.SUCCESS)
        }

        fun error(): JokeApiState {
            return JokeApiState(Status.ERROR)
        }

        fun loading(): JokeApiState {
            return JokeApiState(Status.LOADING)
        }
    }
}

```

JokeResponse.kt:

```

package com.example.jokeapplication.data.network

import kotlinx.serialization.SerialName
import kotlinx.serialization.Serializable

@Serializable
data class JokeResponse(
    @SerialName("category")
    val category: String,
    @SerialName("setup")
    val setup: String,
    @SerialName("delivery")
    val delivery: String
)

```

JokesApi.kt:

```
package com.example.jokeapplication.data.network

import retrofit2.http.GET
import retrofit2.http.Query

interface JokesApi {
    @GET("/joke/Any")
    suspend fun getRandomJoke(
        @Query("blacklistFlags") blacklistFlags: List<String> = listOf(
            "nsfw",
            "religious",
            "political",
            "racist",
            "sexist",
            "explicit"
        ),
        @Query("type") type: String = "twopart",
        @Query("amount") amount: Int = 10
    ): JokesResponse
}
```

JokesResponse.kt:

```
package com.example.jokeapplication.data.network

import kotlinx.serialization.SerialName
import kotlinx.serialization.Serializable

@Serializable
data class JokesResponse(
    @SerialName("jokes")
    val jokes: List<JokeResponse>?=null
)
```

RetrofitInstance.kt:

```
package com.example.jokeapplication.data.network

import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory

object RetrofitInstance {
    private const val BASE_URL = "https://v2.jokeapi.dev/"

    val api: JokesApi by lazy {
        Retrofit.Builder()
            .baseUrl(BASE_URL)
            .addConverterFactory(GsonConverterFactory.create())
            .build()
            .create(JokesApi::class.java)
    }
}
```

Status.kt:

```
package com.example.jokeapplication.data.network
```



```
enum class Status {
    SUCCESS,
    ERROR,
    LOADING
}
```

Joke.kt:

```
package com.example.jokeapplication.data

import androidx.room.Entity
import androidx.room.PrimaryKey
import kotlinx.serialization.SerialName
import kotlinx.serialization.Serializable

@Serializable
@Entity(tableName = "jokes")
data class Joke(
    @SerialName("id")
    @PrimaryKey
    val id: String,
    @SerialName("category")
    val category: String,
    @SerialName("question")
    val question: String,
    @SerialName("answer")
    val answer: String,
    val fromNetwork: Boolean
)
```

JokeGenerator.kt:

```
package com.example.jokeapplication.data

import com.example.jokeapplication.ui.joke_list.JokeViewModel
import java.util.UUID

class JokeGenerator {

    private val dataBase = JokeStorage

    fun addNewJokeStart(joke: Joke) {
        dataBase.data.add(0, joke)
    }

    fun addNewJokeDB(category: String, question: String, answer: String,
        fromNetwork: Boolean) {
        JokeViewModel().addJoke(
            Joke(
                id = UUID.randomUUID().toString(),
                category = category,
                question = question,
                answer = answer,
                fromNetwork = fromNetwork
            )
        )
    }

    fun addNewJokeEnd(category: String, question: String, answer: String,
        fromNetwork: Boolean) {
```

```

        DataBase.data.add(
            Joke(
                id = UUID.randomUUID().toString(),
                category = category,
                question = question,
                answer = answer,
                fromNetwork = fromNetwork
            )
        )
    }
}

```

JokeManager.kt:

```

package com.example.jokeapplication.data

class JokeManager {

    val data = JokeStorage.data

    val generator = JokeGenerator()

    fun isDataNull(): Boolean {
        return data.size == 0
    }

}

```

JokesRepository.kt:

```

package com.example.jokeapplication.data

import com.example.jokeapplication.data.db.JokeDao
import com.example.jokeapplication.data.db.cache.Cache
import com.example.jokeapplication.data.db.cache.CacheDao
import com.example.jokeapplication.data.network.JokeApiState
import com.example.jokeapplication.data.network.JokesApi
import com.example.jokeapplication.data.network.RetrofitInstance
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.firstOrNull
import kotlinx.coroutines.flow.flow
import kotlinx.coroutines.flow.flowOn
import kotlinx.coroutines.withContext
import java.util.UUID

class JokesRepository(private val apiService: JokesApi, private val jokeDao:
JokeDao, private val cacheDao: CacheDao) {

    private val CACHE_TIMEOUT = 24 * 60 * 60 * 1000L

    suspend fun getCache(): List<Cache> {
        return withContext(Dispatchers.IO) {
            val cachedJokes =
cacheDao.getCachedJokes(System.currentTimeMillis() -
CACHE_TIMEOUT).firstOrNull()
            (if (!cachedJokes.isNullOrEmpty()) {
                cachedJokes
            } else {
                try {

```

```

        val jokesDto = RetrofitInstance.api.getRandomJoke().jokes
        val jokes = jokesDto!!.map {
            Cache(
                id = UUID.randomUUID().toString(),
                category = it.category,
                question = it.setup,
                answer = it.delivery,
                fromNetwork = true,
                timestamp = System.currentTimeMillis()
            )
        }
        cacheDao.clearCache()
        cacheDao.insertCache(jokes)
        emptyList()
    } catch (e: Exception) {
        emptyList()
    }
    }) as List<Cache>
}

suspend fun getJokes(): Flow<JokeApiState> {
    return flow {
        emit(JokeApiState.success())
    }.flowOn(Dispatchers.IO)
}

fun getAllJokes(): Flow<List<Joke>> = jokeDao.getAllJokes()

suspend fun addJoke(joke: Joke) {
    jokeDao.insert(joke)
}
}

```

JokeStorage.kt:

```

package com.example.jokeapplication.data

object JokeStorage {

    val data = mutableListOf<Joke>()

}

```

JokeDetailsFragment.kt:

```

package com.example.jokeapplication.ui.joke_details

import android.os.Bundle
import android.view.View
import android.widget.Toast
import androidx.core.os.bundleOf
import androidx.fragment.app.Fragment
import by.kirich1409.viewbindingdelegate.viewBinding
import com.example.jokeapplication.R
import com.example.jokeapplication.data.Joke
import com.example.jokeapplication.data.JokeManager
import com.example.jokeapplication.databinding.FragmentJokeDetailsBinding

class JokeDetailsFragment : Fragment(R.layout.fragment_joke_details) {

```

```

private val binding by viewBinding<FragmentJokeDetailsBinding>::bind

private val dataBase = JokeManager().data

private var jokeID: String? = null

companion object {

    private const val ARG_JOKEID = "jokeID"

    fun newInstance(jokeID: String) =
        JokeDetailsFragment().apply {
            arguments = bundleOf(ARG_JOKEID to jokeID)
        }
}

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    arguments?.let {
        jokeID = it.getString(ARG_JOKEID)
    }
}

override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
    super.onViewCreated(view, savedInstanceState)

    handleExtra()
}

private fun handleExtra() {
    if (jokeID == null) {
        handleError()
    } else {
        val item = jokeID?.let { findJokeByID(it) }

        if (item != null) {
            setupJokeData(item)
        } else {
            handleError()
        }
    }
}

private fun findJokeByID(jokeID: String): Joke? {
    for (joke in dataBase) {
        if (joke.id == jokeID) {
            return joke
        }
    }
    handleError()
    return null
}

private fun setupJokeData(joke: Joke) {
    with(binding) {
        category.text = "Категория: ${joke.category}"
        question.text = "Вопрос: ${joke.question}"
        answer.text = "Ответ: ${joke.answer}"
        if (joke.fromNetwork) {
            fromNetwork.text = "Из сети"
        } else {
            fromNetwork.text = "С устройства"
        }
    }
}

```

```

    }

    private fun handleError() {
        Toast.makeText(activity, "Invalid joke data",
            Toast.LENGTH_SHORT).show()
        activity?.supportFragmentManager
            ?.beginTransaction()
            ?.remove(this)
            ?.commit()
    }
}

```

JokeGenerateFragment.kt:

```

package com.example.jokeapplication.ui.joke_generate

import android.os.Bundle
import android.view.View
import androidx.fragment.app.Fragment
import by.kirich1409.viewbindingdelegate.viewBinding
import com.example.jokeapplication.R
import com.example.jokeapplication.data.JokeManager
import com.example.jokeapplication.databinding.FragmentJokeGenerateBinding

class JokeGenerateFragment : Fragment(R.layout.fragment_joke_generate) {

    private val binding by viewBinding(FragmentJokeGenerateBinding::bind)

    private val generator = JokeManager().generator

    override fun onCreateView(view: View, savedInstanceState: Bundle?) {
        super.onCreateView(view, savedInstanceState)

        binding.saveButton.setOnClickListener {
            addNewJoke()
        }
    }

    private fun addNewJoke() {
        val textCategory = binding.editCategory.text.toString()
        val textQuestion = binding.editQuestion.text.toString()
        val textAnswer = binding.editAnswer.text.toString()
        if (textCategory == "" || textQuestion == "" || textAnswer == "") {
            binding.textError.text = "Нужно заполнить все пустые поля!"
        } else {
            generator.addNewJokeDB(textCategory, textQuestion, textAnswer,
                false)
            parentFragmentManager.popBackStack()
        }
    }
}

```

JokeAdapter.kt:

```

package com.example.jokeapplication.ui.joke_list.recycler.adapter

import android.view.LayoutInflater
import android.view.ViewGroup
import androidx.recyclerview.widget.RecyclerView.Adapter

```

```

import com.example.jokeapplication.data.Joke
import com.example.jokeapplication.databinding.JokeItemBinding
import com.example.jokeapplication.ui.joke_list.JokeListFragment
import com.example.jokeapplication.ui.joke_list.recycler.JokeViewHolder

class JokeAdapter : Adapter<JokeViewHolder>() {

    private var data = emptyList<Joke>()

    fun setData(newData: List<Joke>) {
        data = newData
        notifyDataSetChanged()
    }

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
JokeViewHolder {
        val inflater = LayoutInflater.from(parent.context)
        val binding = JokeItemBinding.inflate(inflater)
        return JokeViewHolder(binding).apply {
            binding.root.setOnClickListener { v ->
                val jokeID = getID(data[adapterPosition])
                JokeListFragment().openDetailsFragment(v, jokeID)
            }
        }
    }

    override fun getItemCount() = data.size

    override fun onBindViewHolder(holder: JokeViewHolder, position: Int) {
        holder.bind(data[position])
    }
}

```

JokeViewHolder.kt:

```

package com.example.jokeapplication.ui.joke_list.recycler

import androidx.recyclerview.widget.RecyclerView
import com.example.jokeapplication.data.Joke
import com.example.jokeapplication.databinding.JokeItemBinding

class JokeViewHolder(
    private val binding: JokeItemBinding
) : RecyclerView.ViewHolder(binding.root) {

    fun bind(joke: Joke) {
        bindCategory(joke.category)
        bindQuestion(joke.question)
        bindAnswer(joke.answer)
        bindFromNetwork(joke.fromNetwork)
    }

    fun bindCategory(category: String) {
        binding.category.text = category
    }

    fun bindQuestion(question: String) {
        binding.question.text = question
    }
}

```

```

fun bindAnswer(answer: String) {
    binding.answer.text = answer
}

fun bindFromNetwork(fromNetwork: Boolean) {
    if (fromNetwork) {
        binding.fromNetwork.text = "Из сети"
    } else {
        binding.fromNetwork.text = "С устройства"
    }
}

fun getID(joke: Joke): String {
    return joke.id
}
}

```

JokeListFragment.kt:

```

package com.example.jokeapplication.ui.joke_list

import android.os.Bundle
import android.view.View
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import androidx.core.view.isVisible
import androidx.fragment.app.Fragment
import androidx.lifecycle.ViewModelProvider
import androidx.lifecycle.LifecycleScope
import androidx.recyclerview.widget.RecyclerView
import by.kirich1409.viewbindingdelegate.viewBinding
import com.example.jokeapplication.R
import com.example.jokeapplication.data.JokeManager
import com.example.jokeapplication.data.JokesRepository
import com.example.jokeapplication.data.db.JokeDatabase
import com.example.jokeapplication.data.db.cache.CacheDatabase
import com.example.jokeapplication.data.network.RetrofitInstance
import com.example.jokeapplication.data.network.Status
import com.example.jokeapplication.databinding.FragmentJokeListBinding
import com.example.jokeapplication.ui.joke_details.JokeDetailsFragment
import com.example.jokeapplication.ui.joke_generate.JokeGenerateFragment
import com.example.jokeapplication.ui.joke_list.recycler.adapter.JokeAdapter
import kotlinx.coroutines.launch

class JokeListFragment : Fragment(R.layout.fragment_joke_list) {

    private val binding: FragmentJokeListBinding by
    viewBinding(FragmentJokeListBinding::bind)

    private val adapter = JokeAdapter()

    private val manager = JokeManager()

    private lateinit var viewModel: JokeViewModel

    private var fromCached: Boolean = false

    override fun onCreateView(view: View, savedInstanceState: Bundle?) {
        super.onCreateView(view, savedInstanceState)
        createRecyclerViewList()

        viewModel = ViewModelProvider(this)[JokeViewModel::class.java]
    }
}

```

```

        binding.recyclerView.addOnScrollListener(object :
RecyclerView.OnScrollListener() {
            override fun onScrolled(recyclerView: RecyclerView, dx: Int, dy:
Int) {
                super.onScrolled(recyclerView, dx, dy)

                if (isScrollToBottom(binding.recyclerView)) {
                    viewModel.fetchNetworkJokes()
                }
            }

            private fun isScrollToBottom(recyclerView: RecyclerView): Boolean
{
                return recyclerView.computeVerticalScrollRange() -
recyclerView.computeVerticalScrollOffset() <=
recyclerView.computeVerticalScrollExtent()
            }
        })

        observeJokes()

        binding.generateJoke.setOnClickListener {
            openGenerateFragment()
        }
    }

    private fun observeJokes() {
        lifecycleScope.launch {
            viewModel.commentState.collect {

                when (it.status) {
                    Status.LOADING -> {
                        binding.shimmerLayout.startShimmer()
                        binding.shimmerLayout.isVisible = true
                        binding.recyclerView.isVisible = false
                    }

                    Status.SUCCESS -> {
                        binding.shimmerLayout.stopShimmer()
                        binding.shimmerLayout.isVisible = false
                        binding.recyclerView.isVisible = true
                        isJokeGenerationRequired()
                    }

                    else -> {
                        binding.shimmerLayout.stopShimmer()
                        binding.shimmerLayout.isVisible = false
                        binding.recyclerView.isVisible = true
                        if (!fromCached) {
                            val jokes = JokesRepository(
                                RetrofitInstance.api,
                                JokeDatabase.INSTANCE.jokeDao(),
                                CacheDatabase.INSTANCE.cacheDao()
                            ).getCache()
                            for (num in jokes) {
                                manager.generator.addNewJokeEnd(
                                    num.category,
                                    num.question,
                                    num.answer,
                                    true
                                )
                            }
                        }
                    }
                }
            }
        }
    }
}

```



```

        }
        if (jokes.isNotEmpty()) {
            Toast.makeText(context, "Load cache jokes",
Toast.LENGTH_SHORT).show()
        } else {
            Toast.makeText(context, "Network error",
Toast.LENGTH_SHORT).show()
        }
        fromCached = true
    }
    isJokeGenerationRequired()
}
}
}

private fun isJokeGenerationRequired() {
    val data = manager.data
    if (manager.isDataNull()) {
        binding.emptyListText.text = "Шутки вышли из чата, добавьте
новую!"
    } else {
        binding.emptyListText.text = ""
        adapter.setNewData(data)
    }
}

private fun openGenerateFragment() {
    val fragment = JokeGenerateFragment()

    requireActivity().supportFragmentManager
        .beginTransaction()
        .replace(R.id.fragment_container, fragment)
        .addToBackStack(null)
        .commit()
}

fun openDetailsFragment(v: View, jokeID: String) {
    val activity = v.context as AppCompatActivity
    val newFragment = JokeDetailsFragment.newInstance(jokeID)
    activity.supportFragmentManager
        .beginTransaction()
        .replace(R.id.fragment_container, newFragment)
        .addToBackStack(null)
        .commit()
}

private fun createRecyclerViewList() {
    binding.recyclerView.adapter = adapter
}
}

```

JokeViewModel.kt:

```

package com.example.jokeapplication.ui.joke_list

//import com.example.jokeapplication.data.db.cache.CacheDatabase
import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.example.jokeapplication.data.Joke
import com.example.jokeapplication.data.JokeManager

```

```

import com.example.jokeapplication.data.JokesRepository
import com.example.jokeapplication.data.db.JokeDatabase
import com.example.jokeapplication.data.db.cache.CacheDatabase
import com.example.jokeapplication.data.network.JokeApiState
import com.example.jokeapplication.data.network.RetrofitInstance
import com.example.jokeapplication.data.network.Status
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.launch

class JokeViewModel : ViewModel() {

    private val manager = JokeManager()

    private val repository: JokesRepository by lazy {
        JokesRepository(
            RetrofitInstance.api,
            JokeDatabase.INSTANCE.jokeDao(),
            CacheDatabase.INSTANCE.cacheDao()
        )
    }

    val commentState = MutableStateFlow(
        JokeApiState(Status.LOADING)
    )

    init {
        fetchNetworkJokes()
    }

    fun fetchNetworkJokes() {
        commentState.value = JokeApiState.loading()
        viewModelScope.launch {
            repository.getJokes()
                .collect {
                    try {
                        val comment = RetrofitInstance.api.getRandomJoke()
                        JokesRepository(RetrofitInstance.api,
JokeDatabase.INSTANCE.jokeDao(),
CacheDatabase.INSTANCE.cacheDao()).getCache()
                        val listJokes = comment.jokes
                        for (num in listJokes!!) {
                            manager.generator.addNewJokeEnd(
                                num.category,
                                num.setup,
                                num.delivery,
                                true
                            )
                        }
                    } catch (e: Exception) {
                        commentState.value = JokeApiState.error()
                    }
                }
            commentState.value = JokeApiState.success()
        }
    }

    fun addJoke(joke: Joke) {
        commentState.value = JokeApiState.loading()
        viewModelScope.launch {
            repository.addJoke(joke)
            JokeManager().generator.addNewJokeStart(joke)
            commentState.value = JokeApiState.success()
        }
    }
}

```

```

    }

    fun loadAllJokes() {
        viewModelScope.launch {
            repository.getAllJokes().collect {
                for (joke in it) {
                    JokeManager().generator.addNewJokeStart(joke)
                }
            }
        }
    }
}

```

App.kt:

```

package com.example.jokeapplication.ui

import android.app.Application
import com.example.jokeapplication.data.db.JokeDatabase
import com.example.jokeapplication.data.db.cache.CacheDatabase

class App: Application() {

    override fun onCreate() {
        super.onCreate()
        JokeDatabase.initDatabase(this)
        CacheDatabase.initDatabase(this)
    }
}

```

MainActivity.kt:

```

package com.example.jokeapplication.ui

import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity
import com.example.jokeapplication.R
import com.example.jokeapplication.databinding.ActivityMainBinding
import com.example.jokeapplication.ui.joke_list.JokeListFragment
import com.example.jokeapplication.ui.joke_list.JokeViewModel

class MainActivity : AppCompatActivity() {

    private lateinit var binding: ActivityMainBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)

        if (savedInstanceState == null) {
            JokeViewModel().loadAllJokes()
            openFragment()
        }
    }

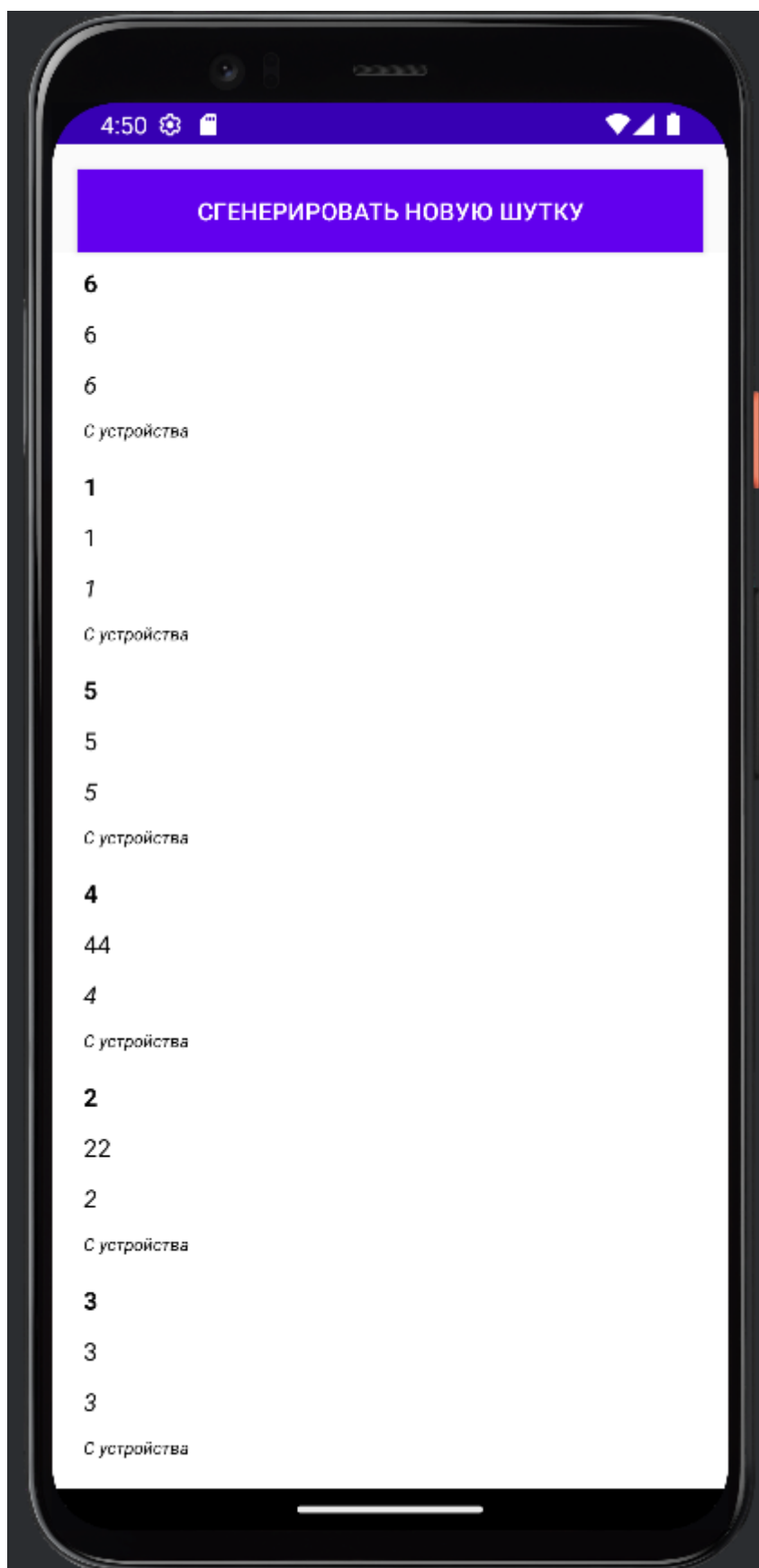
    private fun openFragment() {
        val fragment = JokeListFragment()

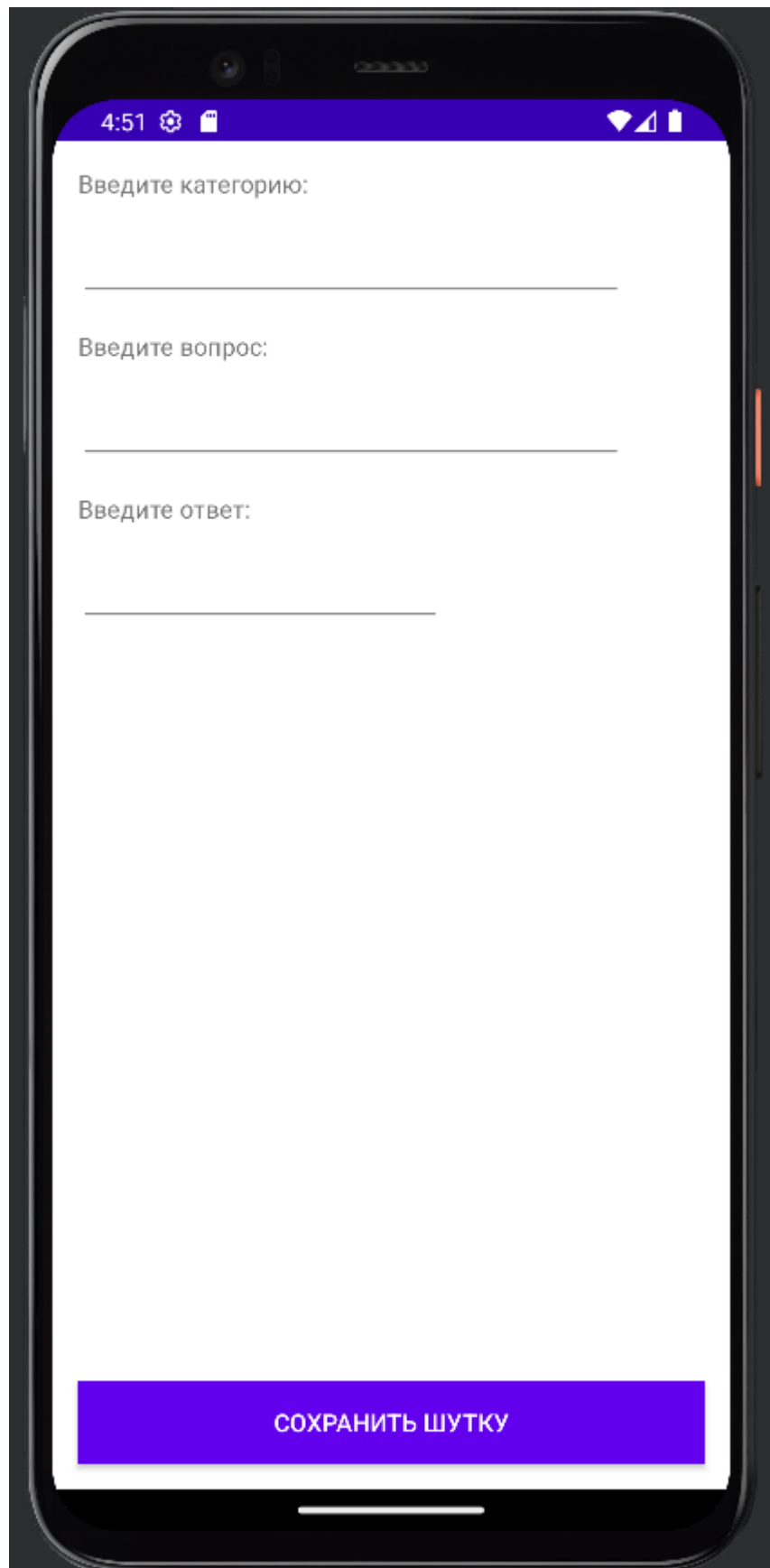
        supportFragmentManager
            .beginTransaction()

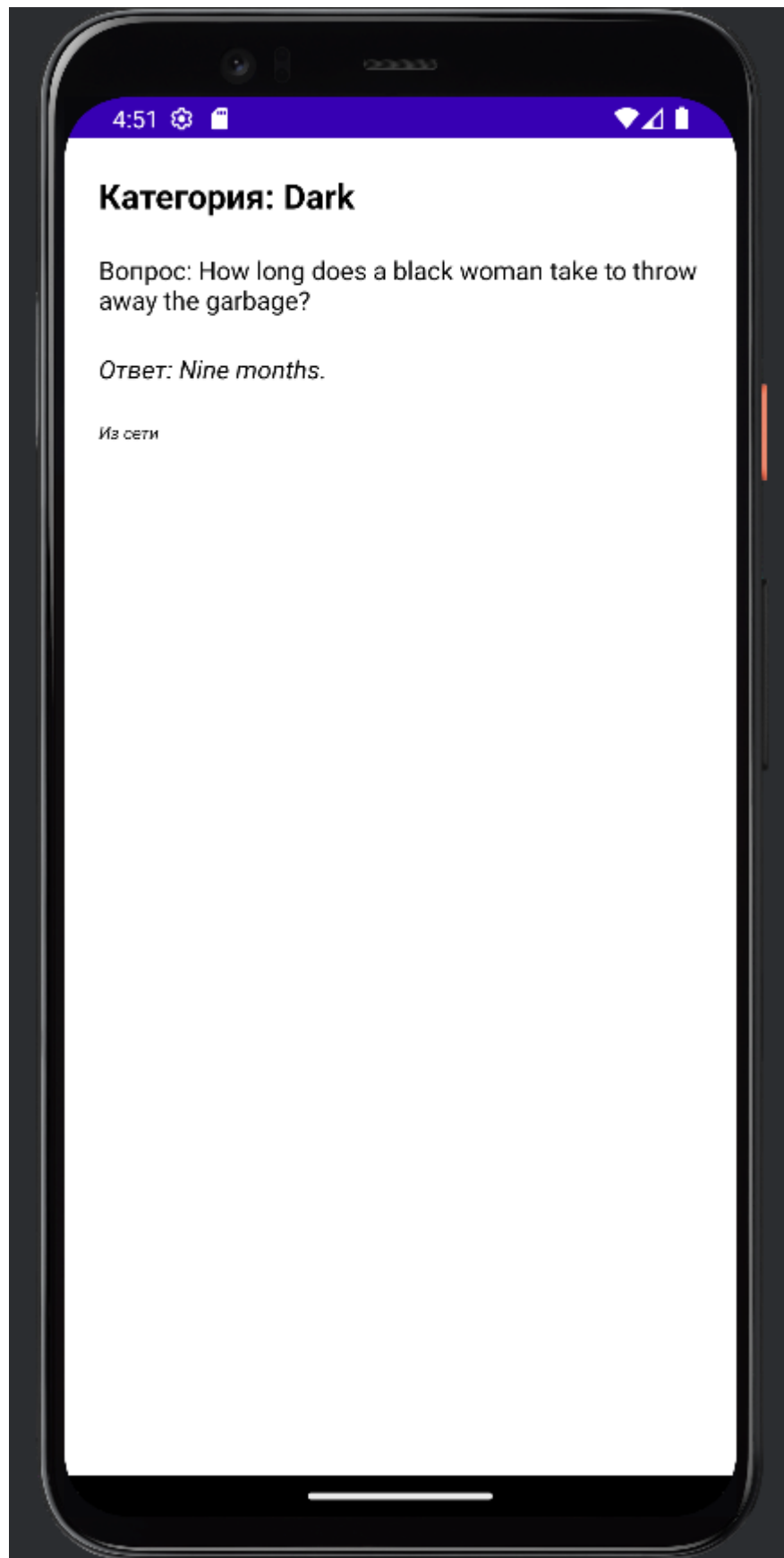
```

```
        .add(R.id.fragment_container, fragment)
        .commit()
    }
}
```

ПРИМЕР ВЫПОЛНЕНИЯ







Вывод: я изучил ранее неизвестный язык программирования и разработал на нем проект.